SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Hallticket num:	2303a54052		
Student Name:	**Mani sai**		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week6 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number: 11.1(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 11 – Data Structures with AI: Implementing Fundamental Structures** **Lab Objectives** • Use AI to assist in designing and implementing fundamental data structures in Python. • Learn how to prompt AI for structure creation, optimization, and documentation. • Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables. • Enhance code quality with AI-generated comments and performance suggestions. **Task Description #1 – Stack Implementation** Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code: class Stack: pass Expected Output: • A functional stack implementation with all required methods and docstrings.	Week6 - Monday

lab 11.1.py > ...

```python
#TASK 01
class Stack:
    """
    A simple Stack data structure implementation using a Python list.
    Supports typical stack operations: push, pop, peek, and is_empty.
    """

    def __init__(self):
        """
        Initializes an empty stack.
        """
        self.items = []

    def push(self, item):
        """
        Adds an item to the top of the stack.

        Args:
            item: The element to be added to the stack.
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/Microsoft
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Top element (peek): 30
Popped element: 30
Is stack empty? False
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

  pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

lab 11.1.py > ...

```python
#TASK 02
class Queue:
    """
    A simple Queue data structure implementation using a Python list.
    Supports typical queue operations: enqueue, dequeue, peek, and size.
    """

    def __init__(self):
        """
        Initializes an empty queue.
        """
        self.items = []

    def enqueue(self, item):
        """
        Adds an item to the end of the queue.

        Args:
            item: The element to be added to the queue.
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/Mi
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Front element (peek): 1
Dequeued element: 1
Queue size: 2
Is queue empty? False
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods.

Sample Input Code:

```
class Node:
    pass

class LinkedList:
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

lab 11.1.py > ...

```python
#TASK 03
class Node:
    """
    A node in a singly linked list.
    """
    def __init__(self, data):
        """
        Initializes a node with given data and next pointer as None.
        Args:
            data: The value to store in the node.
        """
        self.data = data
        self.next = None

class LinkedList:
    """
    A simple singly linked list implementation.
    Supports insertion at the end and displaying the list.
    """
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/Micros
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Linked List: 5 -> 10 -> 15
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

lab 11.1.py > ...

```python
'''
#TASK 04
class BSTNode:
    """
    Node for Binary Search Tree.
    """
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    """
    Binary Search Tree implementation with recursive insert and in-order traversa
    """
    def __init__(self):
        self.root = None

    def insert(self, data):
        """
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Loc
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
In-order Traversal: 10 20 25 30 35
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

    pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

lab 11.1.py > ...

```python
'''
#TASK 05
class HashTable:
    """
    Hash Table implementation using chaining for collision handling.
    Supports insert, search, and delete operations.
    """
    def __init__(self, size=10):
        """
        Initializes the hash table with a given size.
        Args:
            size: Number of buckets in the hash table.
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """
        Computes the hash value for a given key.
        Args:
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/Microsoft/Window
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Search 'banana': 25
Delete 'apple': True
Search 'apple': None
Bucket 0: []
Bucket 1: [('orange', 30)]
Bucket 2: []
Bucket 3: []
Bucket 4: [('banana', 25)]
Bucket 5: []
Bucket 6: []
Bucket 7: []
Bucket 8: []
Bucket 9: []
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

lab 11.1.py > ...

```python
#TASK 06
class Graph:
    """
    Graph implementation using an adjacency list.
    Supports adding vertices, adding edges, and displaying connections.
    """
    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        """
        Adds a vertex to the graph.
        Args:
            vertex: The vertex to add.
        """
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, v1, v2):
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppDat
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
A: ['B', 'C']
B: ['A', 'C']
C: ['A', 'B']
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```python
#Task 07
class PriorityQueue:
    def __init__(self):
        self.heap = []

    def enqueue(self, item, priority):
        # Lower priority value means higher priority
        heapq.heappush(self.heap, (priority, item))

    def dequeue(self):
        if not self.heap:
            return "Priority Queue is empty"
        return heapq.heappop(self.heap)[1]

    def display(self):
        # Show items sorted by priority
        sorted_items = sorted(self.heap)
        print("Priority Queue:", [item for priority, item in sorted_items])

```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/M
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Priority Queue: ['task2', 'task3', 'task1']
Dequeued: task2
Priority Queue: ['task3', 'task1']
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

    pass

Expected Output:

- Insert and remove from both ends with docstrings.

```python
#TASK 08
class DequeDS:
    """
    Double-ended queue (deque) implementation using collections.deque.
    Supports insertion and removal from both ends.
    """

    def __init__(self):
        """
        Initializes an empty deque.
        """
        self.deque = deque()

    def insert_front(self, item):
        """
        Inserts an item at the front of the deque.

        Args:
            item: The element to insert.
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 17

```
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Loca
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Deque: [5, 10, 20]
Removed from front: 5
Removed from rear: 20
Deque: [10]
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding>
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
lab 11.1.py > ...
#TASK 09
# Table mapping feature → chosen data structure → justification
feature_table = [
    {
        "Feature": "Student Attendance Tracking",
        "Data Structure": "Queue",
        "Justification": "A queue is ideal for tracking students entering and exiting the campus in the order of arrival. It ensure
    },
    {
        "Feature": "Event Registration System",
        "Data Structure": "Hash Table",
        "Justification": "A hash table allows for fast search, insertion, and removal of participants by their unique IDs or names.
    },
    {
        "Feature": "Library Book Borrowing",
        "Data Structure": "Binary Search Tree (BST)",
        "Justification": "A BST enables efficient searching, insertion, and deletion of books based on attributes like book ID or d
    },
    {
```

```
PROBLEMS 17   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   SPELL CHECKER 17          Python
PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/Microsoft/WindowsApps/python3.10.exe "c:/Users/manis/OneDrive/Do
cuments/Desktop/AI Assisted Coding/lab 11.1.py"
Feature                     Data Structure       Justification
Student Attendance Tracking    Queue                A queue is ideal for tracking students entering and exiting the campus in the order of arrival. It ensures f
irst-in, first-out (FIFO) processing, which matches the real-world flow of attendance logs.
Event Registration System      Hash Table           A hash table allows for fast search, insertion, and removal of participants by their unique IDs or names. Th
is is essential for efficiently managing event registrations and quick lookups.
Library Book Borrowing         Binary Search Tree (BST) A BST enables efficient searching, insertion, and deletion of books based on attributes like book ID or
due date. This helps in quickly finding available books and managing due dates.
Bus Scheduling System          Graph                A graph is suitable for representing bus routes and stop connections, where stops are nodes and routes are e
dges. It allows for efficient route planning and connectivity checks.
Cafeteria Order Queue          Queue                A queue ensures that students are served in the order they arrive, maintaining fairness and order in process
ing cafeteria orders.
Registered: Alice (ID: 101)
Registered: Bob (ID: 102)
Registered: Charlie (ID: 103)
Search ID 102: Bob
Removed: Alice (ID: 101)
Registered Participants:
```

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
    - Stack
    - Queue
    - Priority Queue
    - Linked List
    - Binary Search Tree (BST)
    - Graph
    - Hash Table
    - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
#Task 10
# Table mapping feature → chosen data structure → justification
ecommerce_feature_table = [
    {
        "Feature": "Shopping Cart Management",
        "Data Structure": "Linked List",
        "Justification": "A linked list allows dynamic addition and removal of products from the cart efficiently. It provides fl
    },
    {
        "Feature": "Order Processing System",
        "Data Structure": "Queue",
        "Justification": "A queue ensures that orders are processed in the order they are placed (FIFO). This maintains fairness
    },
    {
        "Feature": "Top-Selling Products Tracker",
        "Data Structure": "Priority Queue",
        "Justification": "A priority queue efficiently keeps track of products with the highest sales counts at the top. This all
    },
    {
```

```
PROBLEMS 17   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   SPELL CHECKER 17                    Python

PS C:\Users\manis\OneDrive\Documents\Desktop\AI Assisted Coding> & C:/Users/manis/AppData/Local/Microsoft/WindowsApps/python3.10.exe "c:/Users/ma
uments/Desktop/AI Assisted Coding/lab 11.1.py"
Feature                         Data Structure       Justification
Shopping Cart Management        Linked List          A linked list allows dynamic addition and removal of products from the cart efficiently.
exibility in managing items without the overhead of shifting elements, which is ideal for a shopping cart where users frequently add or remove pr
Order Processing System         Queue                A queue ensures that orders are processed in the order they are placed (FIFO). This mair
and order integrity, as the first order placed should be the first to be processed and shipped.
Top-Selling Products Tracker    Priority Queue       A priority queue efficiently keeps track of products with the highest sales counts at th
ws for quick retrieval and updating of the top-selling products as sales data changes.
Product Search Engine           Hash Table           A hash table provides fast lookup, insertion, and deletion of products using product IDs
enables instant access to product details, which is essential for a responsive search experience.
Delivery Route Planning         Graph                A graph is ideal for modeling warehouses and delivery locations as nodes and routes as e
is efficient route planning and optimization algorithms for delivery logistics.
Product added: Laptop (ID: 1001, Price: 1200)
Product added: Smartphone (ID: 1002, Price: 800)
Product added: Headphones (ID: 1003, Price: 150)
Search ID 1002: {'name': 'Smartphone', 'price': 800}
Removed product: Laptop (ID: 1001)
Product Catalog:
```